

Guida allo Sviluppo — Progetto 25

Piano di lavoro, struttura del codice e suddivisione tra due persone

Computational Mathematics for Learning and Data Analysis (646AA)

A.A. 2025/26

Indice

1	Panoramica del progetto	2
1.1	Linguaggio e strumenti consigliati	2
2	Struttura dei file	2
3	Piano di lavoro suddiviso in fasi	3
4	Riepilogo della divisione del lavoro	9
5	Dettagli implementativi	9
5.1	Funzioni comuni (<code>lasso_utils.py</code>)	9
5.2	Pre-calcolo di $X^T X$ e $X^T y$	9
5.3	Criteri di arresto	10
5.4	Suggerimenti numerici	10
6	Checklist finale	10
7	Timeline visuale	11

1 Panoramica del progetto

Il Progetto 25 richiede di:

1. **(M)** Costruire un modello *Extreme Learning Machine* (ELM): una rete neurale a uno strato nascosto con pesi casuali fissi e strato di output appreso tramite un problema LASSO (L_1 -regolarizzato).
2. **(A1)** Implementare l'algoritmo *Iteratively Reweighted Least Squares* (IRLS) per risolvere il LASSO.
3. **(A2)** Implementare un algoritmo della classe *Deflected Subgradient Methods* per risolvere lo stesso problema.
4. Confrontare i due algoritmi in termini di convergenza, accuratezza e efficienza.

Attenzione

Vincoli importanti (dal testo del progetto):

- “No off-the-shelf solvers allowed” per la risoluzione del problema LASSO. Si possono usare librerie per operazioni di base (es. `numpy.linalg.solve` per i sotto-sistemi lineari dell'IRLS).
- Il report deve seguire la struttura indicata nelle istruzioni del corso.
- Il codice deve essere ben organizzato, non tutto in un singolo notebook.

1.1 Linguaggio e strumenti consigliati

Aspetto	Consiglio
Linguaggio	Python (con NumPy/SciPy)
Algebra lineare	<code>numpy.linalg</code> , <code>scipy.linalg</code>
Grafici	<code>matplotlib</code>
Dati	<code>scikit-learn</code> (solo per dataset e confronto)
Versioning	<code>git</code> (repo privato!)
Report	L ^A T _E X

2 Struttura dei file

```
cm_project/
+-- src/
|   +-- elm.py           # Classe ELM (modello M)
|   +-- irls.py          # Algoritmo A1 (IRLS)
|   +-- deflected_subgradient.py # Algoritmo A2
|   +-- lasso_utils.py   # Funzioni comuni (f, grad, ...)
|   +-- linear_solvers.py # Cholesky, CG per IRLS
|   +-- data_generation.py # Generazione/caricamento dati
+-- experiments/
|   +-- experiment_convergence.py # Test convergenza e plot
|   +-- experiment_comparison.py  # Confronto A1 vs A2
|   +-- experiment_scalability.py # Scalabilit 
|   +-- experiment_params.py     # Studio parametri
+-- notebooks/
|   +-- workflow.ipynb          # Notebook riassuntivo
+-- results/
```

```

|   +-- figures/                # Grafici generati
|   +-- tables/                 # Tabelle CSV con risultati
+-- theory/
|   +-- guida_teorica.tex
|   +-- guida_sviluppo.tex
+-- report/
|   +-- report.tex              # Report finale
+-- README.md
+-- requirements.txt

```

3 Piano di lavoro suddiviso in fasi

Il progetto si divide naturalmente in **6 fasi**. Per ciascuna fase indichiamo i task, le dipendenze, e una proposta di suddivisione del lavoro tra **Persona A** e **Persona B**.

Fase 0: Setup e comprensione (1 settimana)

Obiettivi:

- Comprendere a fondo il testo del progetto e la teoria sottostante.
- Impostare l'ambiente di sviluppo e la struttura dei file.
- Scrivere la sezione teorica del report ("Setting the stage").

Task:

1. Leggere il testo del progetto e i riferimenti citati (soprattutto d'Antonio–Frangioni 2009 per i deflected subgradient).
2. Creare il repository git e la struttura dei file.
3. Implementare le funzioni di base in `lasso_utils.py`:
 - $f(w) = \|Xw - y\|_2^2 + \lambda \|w\|_1$
 - Gradiente della parte smooth: $\nabla g(w) = 2X^T(Xw - y)$
 - Subgradiente di f : $\partial f(w)$
 - Metrica di gap/residuo
4. Scrivere le prime sezioni del report (§4.2 delle istruzioni).

Divisione del lavoro: Fase 0

- | | |
|-------------------|---|
| Persona A: | Setup repo, struttura file, implementazione <code>lasso_utils.py</code> , test unitari. |
| Persona B: | Studio teoria, prima bozza sezione teorica del report (definizioni, formulazione del problema, condizioni di ottimalità). |
| Insieme: | Discussione e comprensione del progetto. |

Fase 1: Modello ELM e generazione dati (1 settimana)

Obiettivi:

- Implementare la classe ELM completa.
- Preparare dataset sintetici e reali per i test.

Task:

1. Classe ELM (elm.py):

- `__init__(d, p, activation='sigmoid')`: genera W_1 casuale.
- `transform(X_raw)`: calcola $\sigma(W_1 X_{\text{raw}}^T)$, restituisce la matrice delle feature nascoste.
- `fit(X_hidden, y, solver)`: chiama il solver scelto (IRLS o deflected subgradient).
- `predict(X_raw)`: applica il modello completo.

2. Generazione dati (data_generation.py):

- Dataset sintetici: generare w^* sparso, X casuale, $y = Xw^* + \text{noise}$.
- Variare: dimensione (n, m) , sparsità di w^* , livello di rumore.
- Opzionale: dataset reali (es. da UCI, sklearn).

Suggerimento

Per la generazione dei dati sintetici, è utile conoscere la soluzione ottima w^* (o almeno f^*) per poter valutare il gap. Un modo: generare w^* sparso, calcolare $y = Xw^*$, e usare un solutore esterno (es. `sklearn.linear_model.Lasso`) per ottenere un riferimento f_{ref}^* .

Divisione del lavoro: Fase 1

Persona A: Classe ELM, scelta della funzione di attivazione, test.

Persona B: Generazione dati sintetici, eventuale importazione dati reali, utilità per il calcolo di f_{ref}^* .

Fase 2: Implementazione di A1 — IRLS (2 settimane)

Obiettivi:

- Implementare l'algoritmo IRLS completo.
- Implementare i solutori lineari necessari (Cholesky, CG).
- Testare su piccole istanze.

Task dettagliati:

2a. Solutori lineari (linear_solvers.py):

1. Risoluzione via equazioni normali + Cholesky:

- Input: $Q \succ 0$, b . Output: w tale che $Qw = b$.
- Si può usare `scipy.linalg.cho_factor / cho_solve`, oppure implementare la propria fattorizzazione.
- *Nota:* `numpy.linalg.solve` è permesso come sotto-passo.

2. Gradiente Coniugato (opzionale ma consigliato):

- Implementare CG come descritto nelle slide di NLA, Lecture 5.
- Vantaggioso per grandi n poiché evita di formare $X^T X$ esplicitamente.
- Operazione dominante: prodotto matrice-vettore Qv .

2b. IRLS (irls.py):

1. Implementare l'Algoritmo 1 della guida teorica.
2. Parametri: ε_{thr} , $\varepsilon_{\text{stop}}$, k_{max} , tipo di solver lineare.
3. Registrare ad ogni iterazione: $f(w_k)$, $\|w_{k+1} - w_k\|$, tempo CPU.
4. **Inizializzazione:** $w_0 = (X^T X)^{-1} X^T y$ (minimi quadrati standard) è una buona scelta.

2c. Test e debug:

1. Verificare su un piccolo esempio ($n = 5$, $m = 20$) che IRLS converge.
2. Confrontare la soluzione con `sklearn.linear_model.Lasso` per validare.
3. Verificare che $0 \in \partial f(w^*)$ alla convergenza (controllare le condizioni di ottimalità componentwise).

Divisione del lavoro: Fase 2

Persona A: Solutori lineari (Cholesky + CG), test unitari dei solutori.

Persona B: IRLS completo, integrazione con solutori, test di convergenza e confronto con solutori di riferimento.

Insieme: Debug, revisione codice reciproca.

Attenzione

Aspetti critici dell'IRLS:

- Il threshold ε_{thr} influenza pesantemente la convergenza e la qualità della soluzione. Testare diversi valori (10^{-6} , 10^{-8} , 10^{-10}).
- Se X non ha rango pieno, $X^T X$ è singolare: il termine $2\lambda W_k^T W_k$ regolarizza, ma attenzione al condizionamento.
- Pre-calcolare $X^T X$ e $X^T y$ una volta per tutte (*non* ricalcolarli ad ogni iterazione).

Fase 3: Implementazione di A2 — Deflected Subgradient (2 settimane)

Obiettivi:

- Implementare l'algoritmo deflected subgradient con target level.
- Calibrare i parametri.
- Testare su piccole istanze e confrontare con A1.

Task dettagliati:

3a. Subgradiente puro (baseline):

1. Implementare il subgradiente standard con stepsize decrescente ($\alpha^i = c/i$) come baseline di confronto.
2. Implementare il Polyak stepsize (usando f_{ref}^* dal solutore di riferimento).
3. Implementare il target level stepsize.

3b. Deflected subgradient (deflected_subgradient.py):

1. Implementare l'Algoritmo 2 della guida teorica.
2. Implementare le regole di deflection:
 - Deflection ottimale (??): formula chiusa come proiezione del minimo quadratico su $[0, 1]$.
 - Stepsize-restricted con $\beta^i \leq \gamma^i$.
 - Deflection-restricted.
3. Registrare ad ogni iterazione: $f(w_i)$, $\bar{f}^i$, target level, $\|d^i\|$, tempo CPU.

3c. Formula chiusa per γ^i : Il problema $\min_{\gamma \in [0,1]} \|\gamma g + (1-\gamma)d^-\|^2$ è un problema quadratico univariato in γ :

$$\phi(\gamma) = \gamma^2 \|g\|^2 + 2\gamma(1-\gamma)\langle g, d^- \rangle + (1-\gamma)^2 \|d^-\|^2.$$

Il minimo non vincolato è:

$$\gamma^* = \frac{\|d^-\|^2 - \langle g, d^- \rangle}{\|g\|^2 - 2\langle g, d^- \rangle + \|d^-\|^2} = \frac{\|d^-\|^2 - \langle g, d^- \rangle}{\|g - d^-\|^2}.$$

Poi si proietta su $[0, 1]$: $\gamma^i = \min(1, \max(0, \gamma^*))$.

3d. Test e calibrazione:

1. Testare su piccole istanze con f^* noto.
2. Studiare l'effetto dei parametri δ_0 , ρ , R , β sulla convergenza.
3. Confrontare subgradiente puro vs. deflected subgradient.

Divisione del lavoro: Fase 3

Persona A: Subgradiente puro (baseline), implementazione delle diverse stepsize rules.

Persona B: Deflected subgradient, regole di deflection, formula chiusa per γ .

Insieme: Calibrazione parametri, confronti iniziali A1 vs A2.

Suggerimento

Suggerimenti per il debugging del deflected subgradient:

- Verificare che il record value $\bar{f}^i$ sia sempre $\leq f(w_0)$.
- Verificare che la direzione d^i sia una combinazione convessa di g^i e d^{i-1} (se $\gamma^i \in [0, 1]$).
- Plottare l'evoluzione di γ^i e α^i nel tempo: se $\gamma^i \rightarrow 1$ rapidamente, la deflection non sta aiutando.
- Confrontare con il Polyak stepsize (che richiede f^*) per verificare che il target level sta funzionando.

Fase 4: Esperimenti e analisi (2 settimane)

Obiettivi:

- Condurre esperimenti sistematici.
- Produrre grafici e tabelle significative.
- Analizzare e commentare i risultati.

Esperimenti da condurre:

4a. Convergenza:

- Plottare $f(w_k) - f^*$ (o $\bar{f}^k - f^*$) vs. iterazioni in scala log-lineare.
- Plottare $f(w_k) - f^*$ vs. tempo CPU.
- Verificare il tasso di convergenza (lineare per IRLS, sublineare per deflected subgradient).
- **Scala logaritmica:** essenziale per osservare la convergenza lineare (appare come retta nel grafico log-lineare).

4b. Scalabilità:

- Variare n (dimensione del problema): $n = 50, 100, 500, 1000, 5000$.
- Variare m (numero di campioni): $m = 2n, 5n, 10n$.
- Misurare tempo CPU totale e per iterazione.
- Identificare il massimo n risolvibile in tempo ragionevole.

4c. Effetto dei parametri:

- Per IRLS: effetto di ε_{thr} ($10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}$).
- Per A2: effetto di δ_0, ρ, R , tipo di deflection.
- Effetto di λ (regolarizzazione) sulla sparsità e sulla convergenza.
- Per ELM: effetto di p (numero di neuroni nascosti) e della funzione di attivazione.

4d. Confronto A1 vs A2:

- Confronto iterazioni necessarie per raggiungere $f(w) - f^* \leq \varepsilon$ per diversi ε .
- Confronto tempo CPU per raggiungere la stessa accuratezza.
- Confronto su diverse dimensioni del problema.
- A1 ha iterazioni costose ma poche; A2 ha iterazioni economiche ma molte: chi “vince”?

4e. Validazione:

- Confronto con `sklearn.linear_model.Lasso` per verificare la correttezza del valore f^* .
- Confronto della soluzione w^* (norma della differenza).
- Verifica delle condizioni di ottimalità (??).

Divisione del lavoro: Fase 4

Persona A: Esperimenti di convergenza e scalabilità per A1 (IRLS). Confronto solutori lineari (Cholesky vs CG).

Persona B: Esperimenti di convergenza e calibrazione parametri per A2 (deflected subgradient).

Insieme: Confronto A1 vs A2, grafici finali, analisi dei risultati.

Suggerimento

Per i grafici, seguire le indicazioni del corso (§4.6 delle istruzioni):

- Usare scale logaritmiche per residui e gap.
- La convergenza lineare appare come retta in un grafico log-lineare.
- Pochi grafici significativi sono meglio di molti grafici illeggibili.
- Ogni grafico deve avere un *purpose* chiaro.

Fase 5: Report e consegna (1–2 settimane)

Obiettivi:

- Completare il report seguendo la struttura richiesta.
- Preparare il pacchetto di consegna (codice + dati + risultati + report).

Struttura del report (dalle istruzioni del corso, §4):

1. **Frontespizio:** numero del gruppo, numero del progetto, testo esatto della richiesta.
2. **Descrizione del problema e degli algoritmi (§4.2):**
 - Formulazione del problema (ELM + LASSO).
 - Breve richiamo della teoria necessaria (non ricopiare le slides!).
 - Descrizione degli algoritmi con i dettagli matematici dell'adattamento al problema specifico.
 - Motivazione delle scelte fatte.
3. **Proprietà attese (§4.3):**
 - Risultati di convergenza: quali teoremi si applicano? Le ipotesi sono soddisfatte?
 - Velocità di convergenza attesa.
 - Complessità computazionale.
4. **Setup sperimentale (§4.5):**
 - Descrizione dei dati (sintetici e/o reali).
 - Scelta dei parametri algoritmici.
5. **Risultati sperimentali (§4.6):**
 - Grafici e tabelle con commenti.
 - Confronto tra gli algoritmi.
 - Confronto con solutori di riferimento.

Divisione del lavoro: Fase 5

- Persona A:** Sezioni del report su A1, risultati sperimentali di IRLS, revisione finale codice.
- Persona B:** Sezioni del report su A2, risultati sperimentali del deflected subgradient, README e documentazione.
- Insieme:** Sezioni introduttive e conclusive, revisione incrociata completa, preparazione del pacchetto di consegna.

Attenzione

Dalle istruzioni del corso:

- **Non** copiare testo da slides, libri, internet. Scrivere con parole proprie.
- **Non** scrivere lunghi paragrafi su argomenti non rilevanti (es. applicazioni pratiche del modello, teoria del ML). Concentrarsi sulla parte *matematica e algoritmica*.
- Citare sempre le fonti: “Questo teorema è da [X, Theorem Y]”.
- Si consiglia di inviare il report *parziale* ai docenti per feedback prima di completare tutto.

4 Riepilogo della divisione del lavoro

Fase	Persona A	Persona B	Durata
0	Setup, utilities, test	Teoria, bozza report	1 sett.
1	Classe ELM	Dati sintetici e reali	1 sett.
2	Solutori lineari (Cholesky, CG)	IRLS completo	2 sett.
3	Subgradiente puro (baseline)	Deflected subgradient	2 sett.
4	Esperimenti A1 + scalabilità	Esperimenti A2 + parametri	2 sett.
5	Report A1 + revisione	Report A2 + README	1–2 sett.
Totale stimato			9–10 sett.

Suggerimento

La suddivisione è indicativa. L'aspetto cruciale è che entrambe le persone comprendano *entrambi* gli algoritmi, poiché l'esame orale può vertere su qualsiasi parte del progetto.

5 Dettagli implementativi

5.1 Funzioni comuni (lasso_utils.py)

```
import numpy as np

def f_lasso(X, y, w, lam):
    """Funzione obiettivo: ||Xw - y||^2 + lambda * ||w||_1"""
    residual = X @ w - y
    return np.dot(residual, residual) + lam * np.sum(np.abs(w))

def grad_smooth(X, y, w):
    """Gradiente della parte smooth: 2 X^T (Xw - y)"""
    return 2 * X.T @ (X @ w - y)

def subgradient_f(X, y, w, lam):
    """Un subgradiente di f in w"""
    g_smooth = grad_smooth(X, y, w)
    # Per w_i != 0, sign(w_i); per w_i = 0, scegli 0
    s = np.sign(w) # np.sign(0) = 0
    return g_smooth + lam * s

def optimality_gap(X, y, w, lam, f_star):
    """Gap rispetto all'ottimo (se noto)"""
    return f_lasso(X, y, w, lam) - f_star
```

5.2 Pre-calcolo di $X^T X$ e $X^T y$

Per l'IRLS, pre-calcolare:

```
XtX = X.T @ X      # 0(mn^2), fatto una sola volta
Xty = X.T @ y      # 0(mn), fatto una sola volta
```

Ad ogni iterazione IRLS, il sistema diventa $(X^T X + 2\lambda W_k^2)w = X^T y$, dove W_k^2 è diagonale e va solo aggiunta alla diagonale di $X^T X$.

5.3 Criteri di arresto

Per IRLS:

- Convergenza relativa: $\|w_{k+1} - w_k\| / \max(1, \|w_k\|) < \varepsilon$.
- Numero massimo di iterazioni.
- (Opzionale) Variazione relativa di f : $|f(w_{k+1}) - f(w_k)| / |f(w_k)| < \varepsilon$.

Per Deflected Subgradient:

- Numero massimo di iterazioni (principale).
- Gap stimato $\bar{f}^i - (f_{\text{ref}} - \delta) < \varepsilon$ (poco affidabile in pratica).
- Tempo CPU massimo.

5.4 Suggerimenti numerici

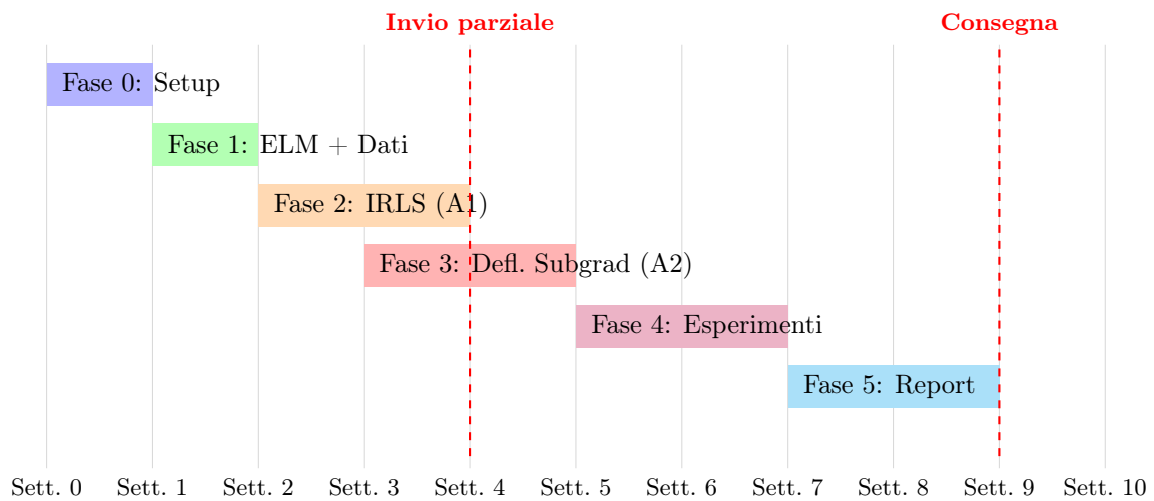
1. **Evitare la formazione esplicita di $X^T X$ se possibile.** Per CG, il prodotto $(X^T X + D)v$ si può calcolare come $X^T(Xv) + Dv$ senza mai formare $X^T X$.
2. **Seed casuale fisso.** Per la riproducibilità, fissare `np.random.seed(42)` (o usare `np.random.RandomState`).
3. **Misura del tempo.** Usare `time.perf_counter()` per misurare i tempi CPU. Non includere il tempo di generazione dei dati o di calcolo delle metriche nel tempo dell'algoritmo.
4. **Warm starting.** Usare la soluzione dell'esecuzione precedente come punto iniziale quando si cambia un parametro (es. λ).
5. **Normalizzazione dei dati.** Normalizzare le colonne di X (media 0, varianza 1) può migliorare il condizionamento e la convergenza.

6 Checklist finale

Prima della consegna, verificare:

- ☐ Il codice gira senza errori partendo da zero (testare su una macchina “pulita”).
- ☐ Il README spiega come eseguire il codice e riprodurre i risultati.
- ☐ I grafici nel report sono leggibili e con scale logaritmiche dove appropriato.
- ☐ Ogni grafico ha un “purpose” chiaro e viene commentato nel testo.
- ☐ Le fonti teoriche sono citate con precisione (numero di teorema/pagina).
- ☐ Le condizioni dei teoremi usati sono discusse (ipotesi soddisfatte o no?).
- ☐ Il confronto con un solutore di riferimento è presente per validazione.
- ☐ I parametri algoritmici sono stati studiati e le scelte sono motivate.
- ☐ Il report è stato inviato parzialmente ai docenti per feedback (consigliato!).
- ☐ Il questionario di valutazione del corso è stato compilato.
- ☐ Il repository **non** è pubblico.

7 Timeline visuale



Suggerimento

Le fasi 2 e 3 sono parzialmente sovrapponibili: A e B possono lavorare in parallelo sui due algoritmi. Si consiglia di inviare il report parziale (sezione teorica + primi risultati di A1) dopo la settimana 4 per ricevere feedback.